UNIT-II
DECISION MAKING & BRANCHING

Decision making with IF Statement:

An **if** statement consists of a Boolean expression followed by one or more statements.

Syntax

The syntax of an 'if' statement in C programming language is −

```
if(boolean_expression) {
  /* statement(s) will execute if the boolean expression is true */
}
```

If the Boolean expression evaluates to **true**, then the block of code inside the 'if' statement will be executed. If the Boolean expression evaluates to **false**, then the first set of code after the end of the 'if' statement (after the closing curly brace) will be executed.

C programming language assumes any **non-zero** and **non-null** values as **true** and if it is either **zero** or **null**, then it is assumed as **false** value.

Flow Diagram

Example

When the above code is compiled and executed, it produces the following result −
a is less than 20;
value of a is : 10

IF- ELSE Statement:

An **if** statement can be followed by an optional **else** statement, which executes when the Boolean expression is false.

Syntax

The syntax of an **if...else** statement in C programming language is −

```
if(boolean_expression) {
  /* statement(s) will execute if the boolean expression is true */
} else {
  /* statement(s) will execute if the boolean expression is false */
}
```

If the Boolean expression evaluates to **true**, then the **if block** will be executed, otherwise, the **else block** will be executed.

C programming language assumes any **non-zero** and **non-null** values as **true**, and if it is either **zero** or **null**, then it is assumed as **false** value.

Flow Diagram

Example

When the above code is compiled and executed, it produces the following result −
a is not less than
20; value of a is :
100

If...else if...else Statement

An **if** statement can be followed by an optional **else if...else** statement, which is very useful to test various conditions using single if...else if statement.
When using if...else if..else statements, there are few points to keep in mind −

- An if can have zero or one else's and it must come after any else if's.
- An if can have zero to many else if's and they must come before the else.
- Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax
The syntax of an if...else if...else statement in C programming language is −

```
if(boolean_expression 1) {
  /* Executes when the boolean expression 1 is true */
} else if( boolean_expression 2) {
  /* Executes when the boolean expression 2 is true */
} else if( boolean_expression 3) {
  /* Executes when the boolean expression 3 is true */
} else {
  /* executes when the none of the above condition is true */
}
```

Example

When the above code is compiled and executed, it produces the following result −
None of the values is matching

Exact value of a is: 100

Nested IF Statement:

It is always legal in C programming to **nest** if-else statements, which means you can use one if or else if statement inside another if or else if statement(s).

Syntax

The syntax for a **nested if** statement is as follows −

```
if( boolean_expression 1) {
  /* Executes when the boolean expression 1 is true */
  if(boolean_expression 2) {
    /* Executes when the boolean expression 2 is true */
  }
}
```

You can nest **else if...else** in the similar way as you have nested *if* statements.

Example

When the above code is compiled and executed, it produces the following result −

Value of a is 100 and b
is 200 Exact value of a
is : 100 Exact value of b
is : 200

Else-If Ladder:

if else if *ladder in C programming* is used to test a series of conditions sequentially. Furthermore, if a condition is tested only when all previous if conditions in the if-else ladder are false. If any of the conditional expressions evaluate to be true, the appropriate code block will be executed, and the entire if-else ladder will be terminated.

Syntax:

```
// any if-else ladder starts with an if statement only if(condition) {

}
else if(condition) {
  // this else if will be executed when condition in if is false and
  // the condition of this else if is true
}
.... // once if-else ladder can have multiple else if else { // at the
end we put else

}
```

Working Flow of the *if-else-if ladder:*

1. The flow of the program falls into the if block.
2. The flow jumps to 1st Condition
3. 1st Condition is tested respectively:
   - If the following Condition yields true, go to Step 4.
   - If the following Condition yields false, go to Step 5.
4. The present block is executed. Goto Step 7.
5. The flow jumps to Condition 2.
   - If the following Condition yields true, go to step 4.
   - If the following Condition yields false, go to Step 6.
6. The flow jumps to Condition 3.
   - If the following Condition yields true, go to step 4.
   - If the following Condition yields false, execute the else block. Goto Step 7.
7. Exits the if-else-if ladder.

if else if ladder *flowchart in C*

Note: *A if-else ladder can exclude else block.*

Example 1: Check whether a number is positive, negative or 0

```
// C Program to check whether

// a number is positive, negative

// or 0 using if else if ladder
#include <stdio.h>


intmain()

{
```

```
intn = 0;



// all Positive numbers will make this

// condition true if
(n > 0) {
      printf("Positive");

}



// all Negative numbers will make this

// condition true else
if(n < 0) {
      printf("Negative");

}



// if a number is neither Positive nor Negative else {
      printf("Zero");

}

return0;

}
```

Output

Zero

Example 2: Calculate Grade According to marks

```
// C Program to Calculate Grade According to marks

// using the if else if ladder
#include <stdio.h>
intmain()

{

    intmarks = 91;

    if(marks <= 100 && marks >= 90)
        printf("A+ Grade");
    elseif(marks    < 90 &&        >= 80)
                    marks
        printf("A
                    Grade");
    elseif(marks    < 80 &&        >   70)
        printf("B   marks          =
    elseif(marks    Grade");
                    < 70 &&
                    marks
                                       60)
                                   >
                                   =
         printf("C  Grade");

      elseif(marks  < 60 &&
                    marks          >= 50)
        printf("D   Grade");


    else

        printf("F Failed");
    return 0;
}
```

Output

A+ Grade

Switch Statement:

A **switch** statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each **switch case**.

Syntax

The syntax for a **switch** statement in C programming language is as follows −

The following rules apply to a **switch** statement −

- The **expression** used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The **constant-expression** for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a **break** statement is reached.
- When a **break** statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a **break**. If no **break** appears, the flow of control will *fall through* to subsequent cases until a break is reached.
- A **switch** statement can have an optional **default** case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No **break** is needed in the default case.

Flow Diagram

Example

When the above code is compiled and executed, it produces the following result −

```
Well done
Your grade
is B
```

Goto Statement:

The **C goto statement** is a jump statement which is sometimes also referred to as an **unconditional jump** statement. The goto statement can be used to jump from anywhere to anywhere within a function.

Syntax:

Syntax1	Syntax2
goto label;	label:
.	.
.	.
.	.
label:	goto label;

In the above syntax, the first line tells the compiler to go to or jump to the statement marked as a label. Here, the label is a user-defined identifier that indicates the target statement. The statement immediately followed after 'label:' is the destination statement. The 'label:' can also appear before the 'goto label;' statement in the above syntax.

Flowchart of goto Statement in C

Below are some examples of how to use a goto statement.

Examples:

Type 1: In this case, we will see a situation similar to as shown in Syntax1 above. Suppose we need to write a program where we need to check if a number is even or not and print accordingly using the goto statement. The below program explains how to do this:

- C

```
// C program to check if a number is

// even or not using goto statement
#include <stdio.h>



// function to check even or not
```

```
voidcheckEvenOrNot(intnum)

{

    if(num % 2 == 0)

        // jump to even
        gotoeven;
    else

        // jump to odd
        gotoodd;



even:

    printf("%d is even", num);

    // return if even
    return;
odd:

    printf("%d is odd", num);

}




intmain()

{

    int num = 26;
    checkEvenOrNot(num);
```

```
    return0;

}
```

Output:
26 is even
Type 2: In this case, we will see a situation similar to as shown in Syntax2 above. Suppose we need to write a program that prints numbers from 1 to 10 using the goto statement. The below program explains how to do this.

- C

```
// C program to print numbers

// from 1 to 10 using goto statement #include
<stdio.h>



// function to print numbers from 1 to 10 void
printNumbers()
{

    intn = 1;
label:
    printf("%d ", n);
    n++;
    if(n <= 10)

        gotolabel;

}
```

```
// Driver program to test above function int
main()
{

    printNumbers();
    return0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10

Disadvantages of Using goto Statement

- The use of the goto statement is highly discouraged as it makes the program logic very complex.
- The use of goto makes tracing the flow of the program very difficult.
- The use of goto makes the task of analyzing and verifying the correctness of programs (particularly those involving loops) very difficult.
- The use of goto can be simply avoided by using break and continue statements.

DECISION MAKING & LOOPING

For Loop:

A **for** loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

Syntax

The syntax of a **for** loop in C programming language is −

```
for ( init; condition; increment ) {
  statement(s);
}
```

Here is the flow of control in a 'for' loop −

- The **init** step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and the flow of control jumps to the next statement just after the 'for' loop.
- After the body of the 'for' loop executes, the flow of control jumps back up to the **increment** statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the 'for' loop terminates.

Flow Diagram

Example

When the above code is compiled and executed, it produces the following result −
value of a:
10 value of
a: 11 value
of a: 12

value of a:
13 value of
a: 14 value
of a: 15
value of a:
16 value of
a: 17 value
of a: 18
value of a:
19

While Loop :

A **while** loop in C programming repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a **while** loop in C programming language is −

```
while(conditio
  n) {
  statement(s);
}
```

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any expression, and true is any nonzero value. The loop iterates while the condition is true.

When the condition becomes false, the program control passes to the line immediately following the loop.

Flow Diagram

Here, the key point to note is that a while loop might not execute at all. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

When the above code is compiled and executed, it produces the following result −
value of a:
10 value of
a: 11

value of a:
12 value of
a: 13 value
of a: 14
value of a:
15 value of
a: 16 value
of a: 17
value of a:
18 value of
a: 19

Do-While Loop:

Unlike **for** and **while** loops, which test the loop condition at the top of the loop, the **do...while** loop in C programming checks its condition at the bottom of the loop.

A **do...while** loop is similar to a while loop, except the fact that it is guaranteed to execute at least one time.

Syntax

The syntax of a **do...while** loop in C programming language is −

```
do {
  statement(s);
} while( condition );
```

Notice that the conditional expression appears at the end of the loop, so the statement(s) in the loop executes once before the condition is tested.

If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop executes again. This process repeats until the given condition becomes false.

Flow Diagram

Example

When the above code is compiled and executed, it produces the following result −

```
value of a:
10 value of
a: 11 value
of a: 12
value of a:
13 value of
a: 14 value
of a: 15
value of a:
16 value of
a: 17 value
of a: 18
value of a:
19
```

Jumps in Loops:

Jump statements in C are used to interrupt the flow of the program or escape a particular section of the program. There are many more operations they can perform within the loops, switch statements, and functions. There are four types of jump statements. The working and execution of the four types of jump statements can be seen in this article.

Scope

- This article covers all the types of jump statements in C language.
- The article is example-oriented, along with the flow charts, syntax, and C program of each jump statement.
- The article covers the advantages and disadvantages of jump statements along with FAQs.

Introduction

Jump Statements interrupt the normal flow of the program while execution and jump when it gets satisfied given specific conditions. The main uses of jump statements are to exit the loops like for, while, do-while also switch case and *executes the given or next block of the code*, *skip the iterations of the loop*, *change the control flow to specific location*, etc.

Types of Jump Statement in C

There are 4 types of Jump statements in C language.

1. Break Statement
2. Continue Statement
3. Goto Statement
4. Return Statement.

Break Statement in C

Break statement exits the loops like for, while, do-while immediately, brings it out of the loop, and starts executing the next block. It also terminates the switch statement.
If we use the break statement in the nested loop, then first, the break statement inside the inner loop will break the inner loop. Then the outer loop will execute as it is, which means the outer loop remains unaffected by the break statement inside the inner loop.

The diagram below will give you more clarity on how the break statement works.

Syntax of Break Statement
The break statement's syntax is simply to use the break keyword.

Syntax:

Flowchart of Break Statement

Flowchart of the break statement in the loop:

The break statement is written inside the loop.

- First, the execution of the loop starts.
- If the condition for the loop is true, then the body of the loop executes; otherwise, the loop terminates.
- If the body of the loop executes, the break condition written inside the body is checked. If it is true, it immediately terminates the loop; otherwise, the loop keeps executing until it meets the break condition or the loop condition becomes false.

Flowchart of the break statement in switch:
The switch case consists of a conditional statement. According to that conditional statement, it has cases. The break statement is present inside every case.

- If the condition inside the case for the conditional statement is true, then it executes the code inside that case.
- After that, it stops the execution of the whole switch block due to encountering a break statement and breaking out of it.

Note: If we don't use break-in cases, all cases following the satisfied case will execute.

So, to better understand, let's go for a C Program. Example: How does Break Statement in C Works? **In a while loop:**
The while loop repeatedly executes until the condition inside is true, so the condition in the program is the loop will execute until the value of a is greater than or equal to 10. But inside the while loop, there is a condition for the break statement: if a equals 3, then break. So the code prints the value of a as 1 and 2, and when it encounters 3, it breaks(terminates the while loop) and executes the print statement outside the while loop.

Code:

Output:

In switch statement:

As the switch statement consists of conditional statements and cases, here we have the conditional statement switch(a), and we have cases where we have different values of a.
The program will take the values of a from the user, and if any value entered by a user matches the value of a inside any case, then the code inside that case will completely execute and will break(terminate the switch statement) to end. If the value of a doesn't match any value of a inside the cases, then the default case will execute.

Code:

Output:

Example: Use of Break Statement in Nested Loop.

In the below code, the inner loop is programmed to perform for 8 iterations, but as soon as the value of j becomes greater than 4 it breaks the inner loop and stops execution and the execution of the outer loop remains unaffected.

Code:

Output:

Example: Checking if the entered number is prime or not using break statement
In the below code, if we find out that the modulo of the number a with a for loop i from 2 to a/2 is zero, then we don't need to check further as we know the number is not prime, so we use break statement to break out from the loop.

Code:

Output:

Continue Statement in C

Continue in jump statement in C skips the specific iteration in the loop. It is similar to the break statement, but instead of terminating the whole loop, it skips the current iteration and continues from the next iteration in the same loop. It brings the control of a program to the beginning of the loop.
Using the continue statement in the nested loop skips the inner loop's iteration only and doesn't affect the outer loop.

Syntax of continue Statement
The syntax for the continue statement is simple continue. It is used below the specified continue condition in the loop.

Syntax:

```
continue;
```

Flow Chart of Continue Statement in C

In this flowchart:

- The loop starts, and at first, it checks if the condition for the loop is true or not.
- If it is not true, the loop immediately terminates.
- If the loop condition is true, it checks the condition for the continue statement.
- If the condition for continue is false, then it allows for executing the body of the loop.

- Otherwise, if the condition for continue is true, it skips the current iteration of the loop and starts with the next iteration.

Uses of Continue Function in C

- If you want a sequence to be executed but exclude a few iterations within the sequence, then continue can be used to skip those iterations we don't need.

Example: How does Continue Statement in C Works?

In the below program, when the continue condition gets true for the iteration i=3 in the loop, then that iteration is getting skips, and the control goes to the update expression of for loop where i becomes 4, and the next iteration starts.

Code:

Output:

Example: C continue statement in nested loops

Rolling 2 dices and count all cases where both dice shows different number:
In the program, the continue statement is used in a nested loop. The outer for loop is for dice1, and the inner for loop is for dice 2. Both the dices are rolling when the program executes since it is a nested loop. If the number of dice1 equals a number of dice 2 then it skips the current iteration, and program control goes to the next iteration of the inner loop.
We know that dice will show the same number 6 times, and in total, there are 36 iterations so that the Output will be 30.

Code:

Output:

```
The dices show different numbers 30 times
```

Return Statement in C
The return statement is a type of jump statement in C which is used in a function to end it or terminate it immediately with or without value and returns the flow of program execution to the start from where it is called.

The function declared with void type does not return any value.

Syntax of Return Statement

Syntax:

Flow Chart of Return Statement

In this flowchart, first, we call another function from the current function, and when that called function encounters a return statement, the flow goes back to the previous function, which we called another function.

How does the return statement works in C?

The return statement is used in a function to return a value to the calling function. The return statement can be written anywhere and often inside the function definition. Still, after the

execution of the first return statement, all the other return statements will be terminated and will be of no use.
The reason why we have the two syntaxes for the return statement is that the return statement can be used in two ways.

- First is that you can use it to terminate the execution of the function when the function is not returning any value.

```
return;
```

- Second, it is to return a value with function calls, and here, the datatype of expression should be the same as the datatype of function.

```
return expression;
```

Here the expression can be any variable, number, or expression. All the return examples given below are valid in C.

But remember, the return statement always returns only one value. The return example given below is valid, but it will only return the rightmost value, i.e., **c**, because we have used a comma operator.

```
return a,b,c;
```

Keeping this in mind, let's see an example to understand the return statement.

Example: Program to understand the return statement

The program below is to calculate the sum of digits 1 to 10. Here first the int sum() block will execute where the logic is written to calculate the digits from 1 to 10 and after calculation, the return statement returns the value of a. Now the program control goes to the main function where the value of a is assigned coming from the sum(); function, and after printing the value of a, the return statement is used to terminate the main function.

Code:

Output:

Example: Using the return statement in void functions

In the code below, the return statement doesn't return anything, and we can see "level2" is not printed as the main function terminates after the return.

Code:

Output:

Examples: Not using a return statement in void return type function

If the return statement inside the void returns any value, it can give errors. As seen in the code, the void Hello() method returns the value 10, whereas it is giving errors in the Output.

Code:

Output:

Advantages and Disadvantages of Jump Statements in C
Advantages

- You can **control the program flow** or alter the follow of the program.
- **Skipping the unnecessary code** can be done using jump statements.
- You can **decide when to break out of the loop** using break statements.
- You **get some sort of flexibility with your code** using jump statements.

Disadvantages

- **Readability of the code is disturbed** because there are jumps from one part of the code to another part.
- **Debugging becomes a little difficult**.
- **Modification of the code becomes difficult.**

Nested Loop:
A **nested loop** means a loop statement inside another loop statement. That is why nested loops are also called "**loop inside loops**". We can define any number of loops inside another loop.

1. Nested for Loop

Nested for loop refers to any type of loop that is defined inside a 'for' loop. Below is the equivalent flow diagram for nested 'for' loops:

Nested for loop in C

Syntax:

```
for ( initialization; condition; increment ) {


    for ( initialization; condition; increment ) {


        // statement of inside loop
    }


    // statement of outer loop
}
```

Example: Below program uses a nested for loop to print a 3D matrix of 2x3x2.

- C

```
// C program to print elements of Three-Dimensional Array

// with the help of nested for loop
#include <stdio.h>



intmain()

{

    // initializing the 3-D array int
    arr[2][3][2]
        = { { { 0, 6 }, { 1, 7 }, { 2, 8 } },

            { { 3, 9 }, { 4, 10 }, { 5, 11 } } };




    // Printing values of 3-D array
```

```
    for(inti = 0; i < 2; ++i) { for(intj
        = 0; j < 3; ++j) {
            for(intk = 0; k < 2; ++k) {

                printf("Element at arr[%i][%i][%i] = %d\n", i,
                    j, k, arr[i][j][k]);
            }

        }

    }

    return0;

}
```

Output

```
Element at arr[0][0][0] = 0
Element at arr[0][0][1] = 6
Element at arr[0][1][0] = 1
Element at arr[0][1][1] = 7
Element at arr[0][2][0] = 2
Element at arr[0][2][1] = 8
Element at arr[1][0][0] = 3
Element at arr[1][0][1] = 9
Element at arr[1][1][0] = 4
Element at arr[1][1][1] = 10
Element at arr[1][2][0] = 5
Element at arr[1][2][1] = 11
```

2. Nested while Loop

A nested while loop refers to any type of loop that is defined inside a ‘while’ loop. Below is the equivalent flow diagram for nested ‘while’ loops:

Nested while loop in C

Syntax:

```
while(condition) {

    while(condition) {

        // statement of inside loop
    }

    // statement of outer loop
}
```

Example: Print Pattern using nested while loops

- C

```
// C program to print pattern using nested while loops
#include <stdio.h>

intmain()

{

    intend = 5;

    printf("Pattern Printing using Nested While loop");

    inti = 1;

    while(i <= end) {
```

```
            printf("\n"); int j
            = 1;
            while (j <= i) {
                  printf("%d ", j); j
                  = j + 1;
            }

            i = i + 1;

      }

      return0;

}
```

Output

Pattern Printing using Nested While loop 1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

3. Nested do-while Loop

A nested do-while loop refers to any type of loop that is defined inside a do-while loop. Below is the equivalent flow diagram for nested ‘do-while’ loops:

do while loop in C

Syntax:

```
do{


    do{


        // statement of inside loop
    }while(condition);


    // statement of outer loop
}while(condition);
```

Note: *There is no rule that a loop must be nested inside its own type. In fact, there can be any type of loop nested inside any type and to any level.*

Syntax:

```
do{


    while(condition) {


        for ( initialization; condition; increment ) {


            // statement of inside for loop
        }


        // statement of inside while loop
    }


    // statement of outer do-while loop
}while(condition);
```

Example: Below program uses a nested for loop to print all prime factors of a number.

```
// C Program to print all prime factors

// of a number using nested loop



#include <math.h>
#include<stdio.h>



// A function to print all prime factors of a given number n void
primeFactors(int n)
{

    // Print the number of 2s that divide n while
    (n % 2 == 0) {
        printf("%d ", 2); n
        = n / 2;
    }




    // n must be odd at this point. So we can skip

    // one element (Note i = i +2)

    for(inti = 3; i <= sqrt(n); i = i + 2) {

        // While i divides n, print i and divide n while(n
        % i == 0) {
```

```
            printf("%d ", i); n
            = n / i;
        }

    }



    // This condition is to handle the case when n

    // is a prime number greater than 2 if(n
    > 2)
        printf("%d ", n);

}



/* Driver program to test above function */ int
main()
{

    int n = 315;
    primeFactors(n);
    return 0;
}
```

Output:
3 3 5 7

Break Inside Nested Loops

Whenever we use a break statement inside the nested loops it breaks the innermost loop and program control is inside the outer loop.

Example:

- C

```
// C program to show working of break statement

// inside nested for loops #include
<stdio.h>



intmain()

{



   inti = 0;

for(inti = 0; i < 5; i++) { for(intj
    = 0; j < 3; j++) {
        // This inner loop will break when i==3 if(i
        == 3) {
            break;

        }

        printf("* ");

    }

    printf("\n");

}

return0;
```

```
}
```

Output

```
* * *
* * *
* * *


* * *
```

In the above program, the first loop will iterate from 0 to 5 but here if i will be equal to 3 it will break and will not print the * as shown in the output.

Continue Inside Nested loops

Whenever we use a continue statement inside the nested loops it skips the iteration of the innermost loop only. The outer loop remains unaffected.

Example:

- C

```
// C program to show working of continue statement

// inside nested for loops #include
<stdio.h>



intmain()

{



    inti = 0;

    for(inti = 0; i < 5; i++) { for(intj
        = 0; j < 3; j++) {
```

```
            // This inner loop will skip when j==2 if
            (j==2) {
                continue;

            }

            printf("%d ",j);

        }

        printf("\n");

    }

    return0;

}
```

In the above program, the inner loop will be skipped when j will be equal to 2. The outer loop will remain unaffected.

UNIT-III FUNCTIONS

Standard Mathematical Functions:

C Programming allows us to perform mathematical operations through the functions defined in <math.h> header file. The <math.h> header file contains various methods for performing mathematical operations such as sqrt(), pow(), ceil(), floor() etc.

C Math Functions

There are various methods in math.h header file. The commonly used functions of math.h header file are given below.

No.	Function	Description
1)	ceil(number)	rounds up the given number. It returns the integer value which is greater than or equal to given number.
2)	floor(number)	rounds down the given number. It returns the integer value which is less than or equal to given number.
3)	sqrt(number)	returns the square root of given number.
4)	pow(base, exponent)	returns the power of given number.
5)	abs(number)	returns the absolute value of given number.

C Math Example

Let's see a simple example of math functions found in math.h header file.

```
1. #include<stdio.h>
2. #include <math.h>
3. int main(){
4. printf("\n%f",ceil(3.6));
5. printf("\n%f",ceil(3.3));
6. printf("\n%f",floor(3.6));
```

```
7.  printf("\n%f",floor(3.2));
8.  printf("\n%f",sqrt(16));
9.  printf("\n%f",sqrt(7));
10. printf("\n%f",pow(2,4));
11. printf("\n%f",pow(3,3));
12. printf("\n%d",abs(-12));
13.  return 0;
14. }
```

Output:

Input/Output:

Unformatted & Formatted I/O Function in C:

Formatted I/O Functions

Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers. These I/O supports all data types like int, float, char, and many more.

Why they are called formatted I/O?

These functions are called formatted I/O functions because we can use format specifiers in these functions and hence, we can format these functions according to our needs.

List of some format specifiers-

SNO.	Format Specifier	Type	Description
1	%d	int/signed int	used for I/O signed integer value

2	%c	char	Used for I/O character value
3	%f	float	Used for I/O decimal floating-point value
4	%s	string	Used for I/O string/group of characters
5	%ld	long int	Used for I/O long signed integer value
6	%u	unsigned int	Used for I/O unsigned integer value
7	%i	unsigned int	used for the I/O integer value
8	%lf	double	Used for I/O fractional or floating data
9	%n	prints	prints nothing

The following formatted I/O functions will be discussed in this section-

1. printf()
2. scanf()
3. sprintf()
4. sscanf()

printf():

printf() function is used in a C program to display any value like float, integer, character, string, etc on the console screen. It is a pre-defined function that is already declared in the stdio.h(header file).

Syntax 1:

To display any variable value.

printf("Format Specifier", var1, var2, …., varn);

Example:

C

```
// C program to implement

// printf() function
#include <stdio.h>



// Driver code
int main()
{

    // Declaring an int type variable int
    a;



    // Assigning a value in a variable a =
    20;



    // Printing the value of a variable printf("%d",
    a);



    return 0;

}
```

Output

20

Syntax 2:

To display any string or a message